

Linguaggi Formali e Compilatori (Formal Languages and Compilers)

prof. S. Crespi Reghizzi, prof. Angelo Morzenti
(prof. Luca Breveglieri)

Prova scritta - 5 febbraio 2013 - Parte I: Teoria

CON SOLUZIONI - A SCOPO DIDATTICO LE SOLUZIONI SONO MOLTO ESTESE E COMMENTATE VARIAMENTE - NON SI RICHIEDE CHE IL CANDIDATO SVOLGA IL COMPITO IN MODO ALTRETTANTO AMPIO, BENSÌ CHE RISPONDA IN MODO APPROPRIATO E A SUO GIUDIZIO RAGIONEVOLE
AVVISO - A PARTIRE DALLA SESSIONE ESTIVA (GIUGNO 2013), GLI ESERCIZI DI ANALISI SINTATTICA ANDRANNO AFFRONTATI SECONDO LA TEORIA *ELR / ELL* - LA TEORIA *LR / LL* NON SARÀ PIÙ AMMESSA - SI VEDA IL SITO WEB DEL CORSO PER MATERIALE DIDATTICO E INFORMAZIONI

NOME:

MATRICOLA:

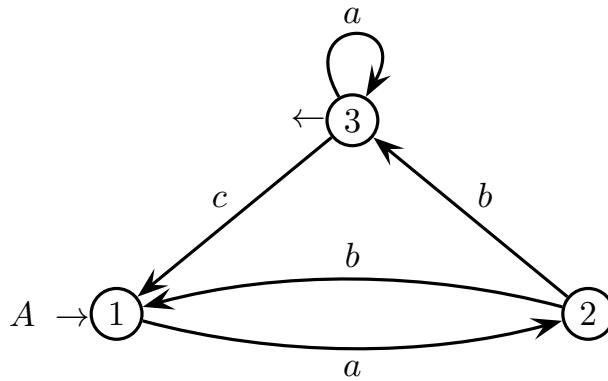
FIRMA:

ISTRUZIONI - LEGGERE CON ATTENZIONE:

- L'esame si compone di due parti:
 - I (80%) Teoria:
 1. espressioni regolari e automi finiti
 2. grammatiche libere e automi a pila
 3. analisi sintattica e parsificatori
 4. traduzione sintattica e analisi semantica
 - II (20%) Esercitazioni Flex e Bison
- Per superare l'esame l'allievo deve sostenere con successo entrambe le parti (I e II), in un solo appello oppure in appelli diversi, ma entro un anno.
- Per superare la parte I (teoria) occorre dimostrare di possedere conoscenza sufficiente di tutte le quattro sezioni (1-4), rispondendo alle domande obbligatorie.
- È permesso consultare libri e appunti personali.
- Per scrivere si utilizzi lo spazio libero e se occorre anche il tergo del foglio; è vietato allegare nuovi fogli o sostituirne di esistenti.
- Tempo: Parte I (teoria): 2h.15m - Parte II (esercitazioni): 60m

1 Espressioni regolari e automi finiti 20%

1. Dato l'alfabeto ternario $\{ a, b, c \}$, si consideri l'automa nondeterministico A seguente:

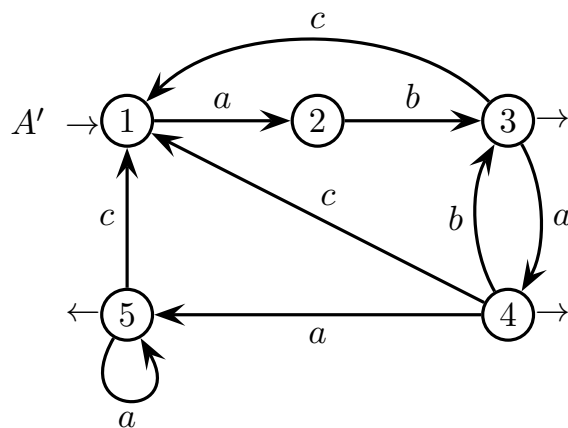
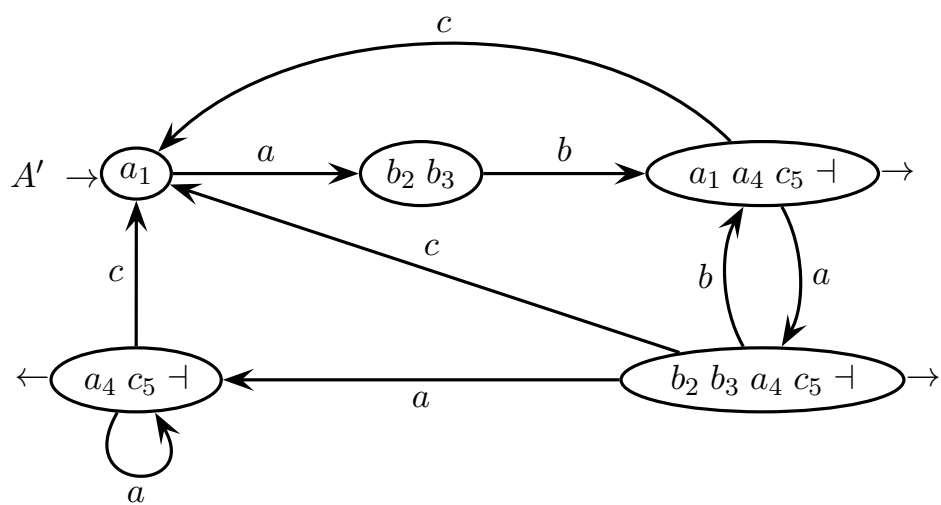
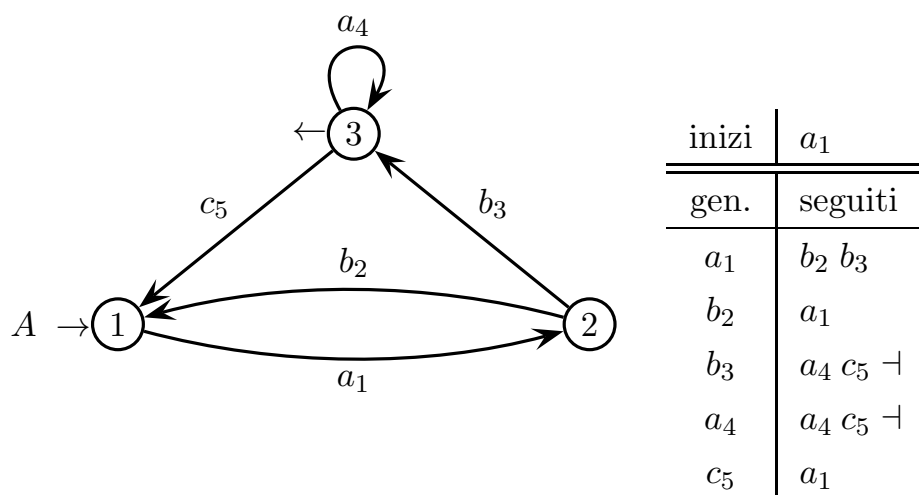


Si risponda alle domande seguenti:

- (a) Tramite il metodo di Berry-Sethi, dall'automa A si ricavi un automa deterministico e minimo A' equivalente.
 - (b) Tramite un metodo a scelta (p. es. eliminazione dei nodi o equazioni linguistiche), dall'automa A si ricavi un'espressione regolare R equivalente.
 - (c) (facoltativa) La lettera b viene cancellata dalle stringhe del linguaggio (p. es. la transizione $2 \xrightarrow{b} 3$ diventa $2 \xrightarrow{\varepsilon} 3$, etc); il resto è invariato. Si trovi alla svelta l'automa deterministico (minimo o no).
-

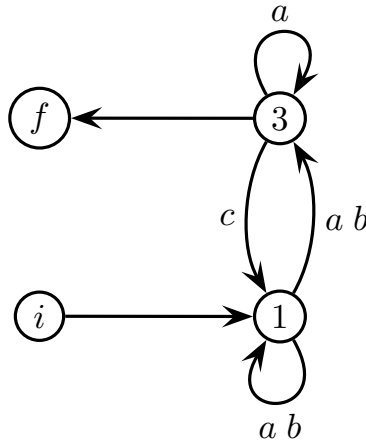
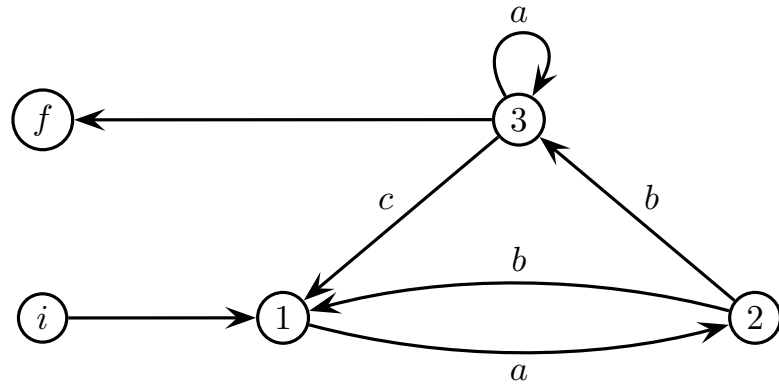
Soluzione

(a) Ecco come ricavare l'automa deterministico A' , tramite Berry-Sethi (BS):

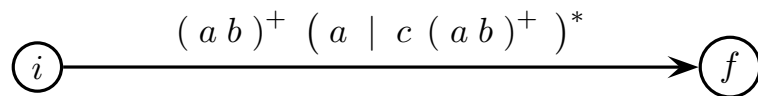
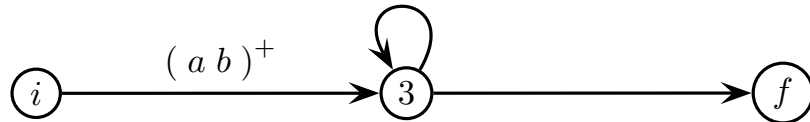


L'automa deterministico di BS A' è in forma ridotta (tutti gli stati sono raggiungibili e definiti) e minimo: i due stati non finali 1 e 2 differiscono per l'arco b , dunque sono distinguibili; lo stato finale 4 differisce dagli altri due finali 3 e 5 per l'arco b , dunque è distinguibile da essi; e con l'arco a gli stati finali 3 e 5 vanno in due stati distinguibili, dunque sono distinguibili. Non ci sono altre possibilità, dunque l'automa A' è minimo.

(b) Ecco come ricavare l'espressione regolare R , tramite eliminazione dei nodi:



$$a \mid c (a b)^+$$



Pertanto ecco una forma possibile per l'espressione regolare R :

$$R = (a b)^+ (a \mid c (a b)^+)^*$$

Beninteso, eliminando nodi in ordine diverso si potrebbero ottenere soluzioni differenti per R , ma equivalenti. Per esempio, anche senza eliminare nodi in ordine diverso ma ricorrendo alla nota trasformazione $(\alpha \mid \beta)^* = \alpha^* (\beta \alpha^*)^*$ (non ambigua), si avrebbe per R la forma equivalente seguente:

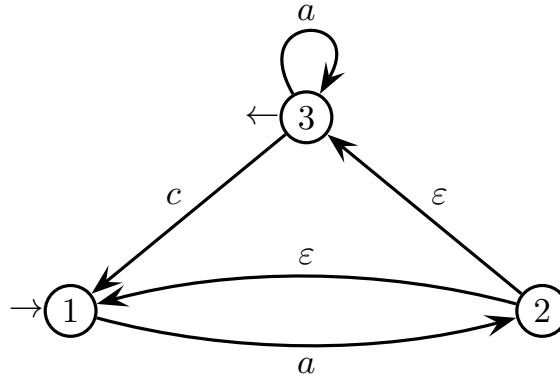
$$R = (a b)^+ a^* (c (a b)^+ a^*)^*$$

che non contiene l'operatore di unione; oppure, ricorrendo alla nota trasformazione $(\alpha \mid \beta)^* = (\alpha^* \beta^*)^*$ (che è ambigua), si avrebbe:

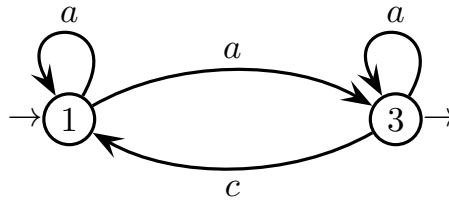
$$R = (a b)^+ \left(a^* (c (a b)^+)^* \right)^*$$

che non ha unione ed è molto compatta (sebbene con tre livelli di iterazione invece di solo due); però è ambigua, mentre le altre due non lo sono.

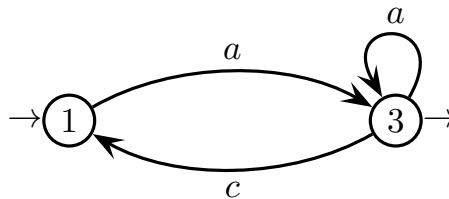
(c) È come dire che l'automa diventa questo, ovviamente ancora nondeterministico:



Tagliando intuitivamente le ε -transizioni (o usando le regole di taglio), si ottiene:



Così è ambiguo: le stringhe a^n ($n \geq 2$) sono riconosciute sia ciclando su 1 sia su 3 (o su entrambi se $n \geq 3$). Però basta togliere l'autoanello su 1 per risolvere l'ambiguità senza alterare il linguaggio:



Ora l'automa è deterministico e perfino minimo.

Beninteso, si potrebbe procedere riapplicando Berry-Sethi, bla bla bla ...

2 Grammatiche libere e automi a pila 20%

1. Si vogliono definire le grammatiche di certe estensioni del classico linguaggio libero $a^n c b^n$ ($n \geq 1$). Si risponda pertanto alle domande seguenti:

- (a) Le stringhe di un linguaggio L contengono un numero pari ≥ 4 di gruppi non vuoti di lettere; ogni gruppo contiene solo lettere a oppure solo lettere b ; e i gruppi sono separati da lettere c isolate (si noti che le c sono in numero dispari). I gruppi di a e di b si alternano, iniziando con un gruppo di a seguito (dopo una c) da un gruppo di b seguito (dopo una c) da un gruppo di a , e così via fino all'ultimo gruppo, formato da b . I gruppi di a e di b che si trovano in posizione simmetrica rispetto alla c centrale, ossia il primo gruppo e l'ultimo, il secondo e il penultimo, etc, hanno la stessa lunghezza.

Esempio di stringa $\in L$:

$$a^2 c b^3 c a^3 c b^2$$

Esempi di stringhe $\notin L$:

$$a^3 c b^2 c a^3$$

$$a^2 c b^3 c a^2 c b^2$$

Si scriva, se possibile, una grammatica BNF (non estesa) per il linguaggio L .

- (b) Si consideri il linguaggio L_1 variante di L , le cui stringhe contengono gruppi di a e di b . Il numero di gruppi in L_1 è lo stesso che in L (pari e ≥ 4) ed essi si alternano come in L , ma in modo che la lunghezza di ciascun gruppo di a sia strettamente maggiore di quella del corrispondente gruppo di b che occupa la posizione simmetrica rispetto alla lettera c centrale; ossia il primo gruppo è più lungo dell'ultimo, il penultimo è più lungo del secondo, etc.

Esempio di stringa $\in L_1$:

$$a^3 c b c a^2 c b^2$$

Esempi di stringhe $\notin L_1$:

$$a^2 c b^2 c a^3 c b^2$$

$$a^3 c b^3 c a^2 c b^2$$

Si scriva, se possibile, una grammatica BNF (non estesa) per il linguaggio L_1 .

- (c) (facoltativa) Si consideri una seconda variante L_2 del linguaggio L , le cui stringhe contengono gruppi di a e di b . Il numero di gruppi in L_2 è lo stesso che in L (pari e ≥ 4) ed essi si alternano come in L , ma in modo che tutti i gruppi di a e di b abbiano esattamente la stessa lunghezza.

Esempio di stringa $\in L_2$:

$$a^3 c b^3 c a^3 c b^3$$

Si scriva, se possibile, una grammatica BNF (non estesa) per il linguaggio L_2 .

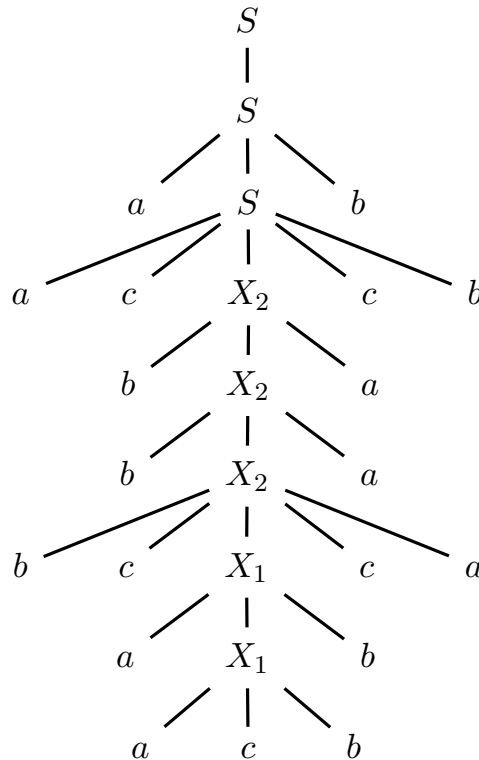
Soluzione

(a) Ecco una possibile grammatica G per il linguaggio L (assioma S):

$$G \left\{ \begin{array}{l} S \rightarrow a S b \mid a c X_2 c b \\ X_1 \rightarrow a X_1 b \mid a c X_2 c b \mid a c b \\ X_2 \rightarrow b X_2 a \mid b c X_1 c a \mid b c a \end{array} \right.$$

Qui i nonterminali X_1 e X_2 generano i gruppi simmetrici di lettere $a^n \dots b^n$ e $b^n \dots a^n$ (con $n \geq 1$), rispettivamente, mediante regole con la familiare struttura $X \rightarrow a X b \mid a b$, e viceversa scambiando a e b ; inoltre essi generano una lettera c separatrice ogniqualvolta si alternano (o terminano). Sia X_1 sia X_2 generano due gruppi (uno di a e uno di b) che quindi saranno sempre in numero pari. Il nonterminale S genera il primo gruppo simmetrico di lettere $a^n \dots b^n$, allo stesso modo di X_1 , tranne che S non può terminare senza anche generare un secondo gruppo simmetrico $b^n \dots a^n$, e quindi obbligando il numero di gruppi ad essere ≥ 4 .

L'albero sintattico della stringa $a^2 c b^3 c a^2 c b^2 c a^3 c b^2$ (che ha 6 gruppi) aiuta a capire:

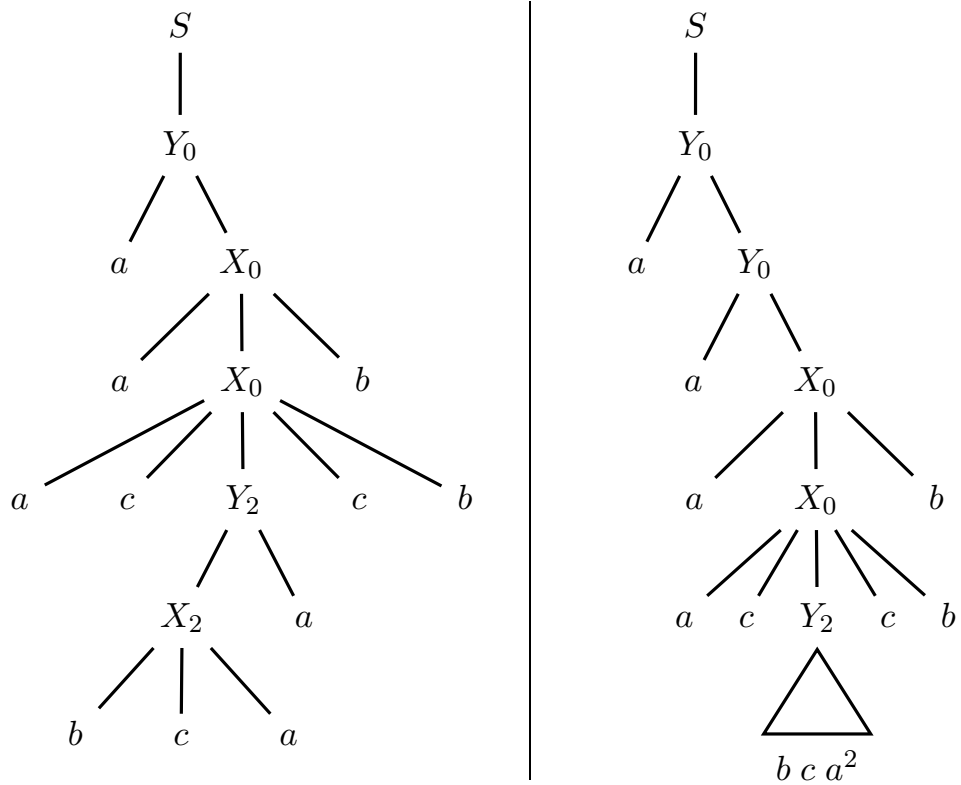


Si vede che si inizia con S e poi i nonterminali X_2 e X_1 si alternano, generando i gruppi $a^m b^m$ e $b^n a^n$ mediante la familiare regola autoinclusiva.

(b) Ecco una possibile grammatica G_1 per il linguaggio L_1 (assioma S):

$$G_1 \left\{ \begin{array}{l} S \rightarrow Y_0 \\ \hline Y_0 \rightarrow a Y_0 \mid a X_0 \\ X_0 \rightarrow a X_0 b \mid a c Y_2 c b \\ \hline Y_1 \rightarrow a Y_1 \mid a X_1 \\ X_1 \rightarrow a X_1 b \mid a c Y_2 c b \mid a c b \\ \hline Y_2 \rightarrow Y_2 a \mid X_2 a \\ X_2 \rightarrow b X_2 a \mid b c Y_1 c a \mid b c a \end{array} \right.$$

Qui i nonterminali X_1 e X_2 funzionano come nella grammatica G , ma alternandosi passano per i nonterminali Y_1 e Y_2 , rispettivamente; i due nuovi nonterminali aggiungono un numero ≥ 1 di a ai gruppi rispettivi; così vengono ottenuti i rapporti di lunghezza richiesti. Il vincolo del numero di gruppi ≥ 4 viene ottenuto allo stesso modo, tramite la coppia di nonterminali Y_0 e X_0 che funzionano come Y_1 e X_1 rispettivamente, ma non possono terminare la generazione della stringa. Gli alberi sintattici della grammatica G_1 somigliano a quelli di G , salvo aggiungere i nodi Y , le cui regole ricorsive sono tutte lineari (a destra o sinistra). L'albero sintattico (a sx) della stringa di esempio $a^3 c b c a^2 c b^2$, aiuta a capire:

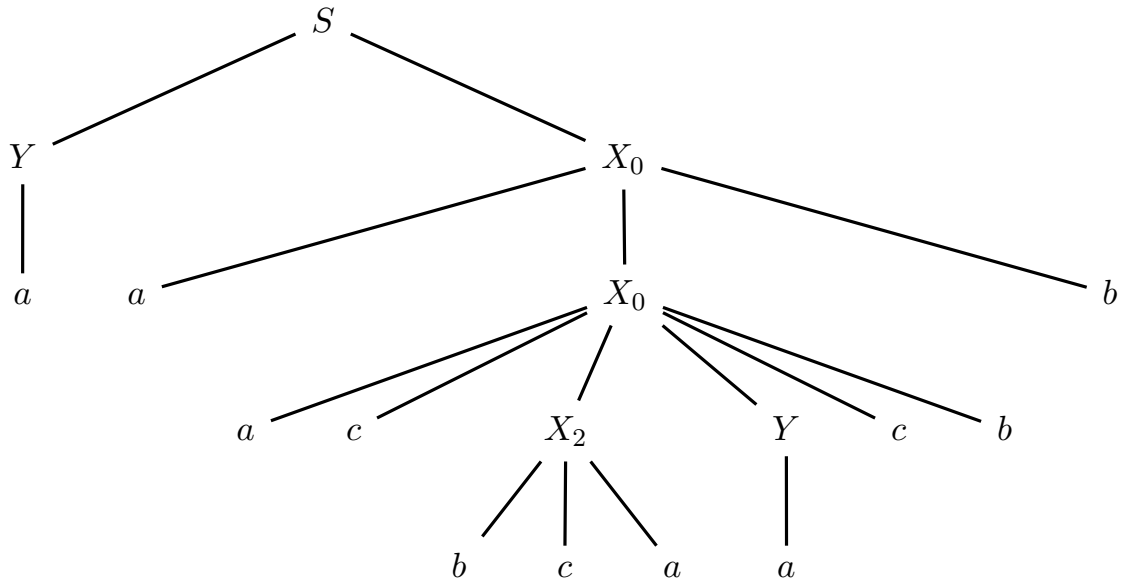


A sx non figurano le regole lineari $Y_1 \rightarrow a Y_1$ e $Y_2 \rightarrow Y_2 a$, ma è chiaro dove si potrebbero inserire e con quale effetto: aumentare ancora le a rispetto alle b ; a dx si veda l'abbozzo di albero per la stringa $a^4 c b c a^2 c b^2$ (similmente con Y_2).

Ecco peraltro una grammatica variante G'_1 (assioma S), equivalente a G_1 :

$$G'_1 \left\{ \begin{array}{l} S \rightarrow Y X_0 \\ X_0 \rightarrow a X_0 b \mid a c X_2 Y c b \\ X_1 \rightarrow a X_1 b \mid a c X_2 Y c b \mid a c b \\ X_2 \rightarrow b X_2 a \mid b c Y X_1 c a \mid b c a \\ Y \rightarrow a Y \mid a \end{array} \right.$$

Qui la generazione dei terminali a in eccesso ai corrispondenti terminali b , è svolta da un solo nonterminale Y con regola lineare, il quale affianca (risp. a sx e dx) i nonterminali X_1 e X_2 che si alternano nella generazione dei gruppi. Ecco un albero sintattico per la stringa di esempio $a^3 c b c a^2 c b^2$ (la stessa di prima):



La grammatica G'_1 ha meno nonterminali di G_1 , ma regole più lunghe; e ha alberi sintattici meno profondi, ma più larghi. Qui non si vede la regola lineare $Y \rightarrow a Y$, ma si capisce subito dove essa si inserisce e con quale effetto.

- (c) Il linguaggio L_2 è composto da stringhe con quattro gruppi di lunghezza identica e chiaramente è impossibile definirlo mediante una grammatica libera BNF (si pensi al noto linguaggio non libero $a^n b^n c^n$), come neppure mediante una estesa.

2. Ecco parte della definizione ufficiale del linguaggio Lua, che è di uso frequente nel programmare (video)giochi. La sintassi è in forma *EBNF* (assioma **stat**); le parentesi graffe indicano che un costrutto può mancare o essere ripetuto una o più volte.

```
stat    ::= varlist '=' explist
varlist ::= var {',' var}
explist ::= {exp ','} exp
exp     ::= nil | false | true | Number | String | exp binop exp
          | unop exp
binop   ::= '+' | '-' | '*' | '/' | '^' | '..' | '<' | '=='
          | and | or
unop    ::= '-' | not | '#'
```

Lua supports the usual arithmetic operators: the binary '+' (addition), '-' (subtraction), '*' (multiplication) and '/' (division); '^' (exponentiation); and the unary '-' (negation).
The relational operators to be considered are: '==' and '<'.
The logical operators are: 'and', 'or' and 'not'.
The string concatenation operator is denoted by two dots '..'.
The length (of string) operator is denoted by the unary operator '#'.
Some examples of statement:

```
x = 10 or 20 and - y
y, z = nil and (10 or x), 7 + #'abc' * w^3
```

La grammatica data sopra è ambigua e non assegna precedenze agli operatori, ma il manuale di Lua le specifica separatamente e inoltre indica quali operatori sono associativi a destra e quali associativi a sinistra, come segue:

Operator precedence follows the table below from lower to higher priority:

or	
and	
<	==
..	
+	-
*	/
not	# - (unary)
^	

As usual, you can use parentheses to change the precedences of an expression. The concatenation '..' and exponentiation '^' operators are right associative. All other binary operators are left associative.

Modificando la grammatica data, si scriva una grammatica in forma non estesa per permettere l'uso di parentesi, assegnare agli operatori le precedenze indicate sopra, e assegnare agli operatori le associazioni da sinistra o da destra specificate sopra.

Soluzione

La grammatica richiesta è questa (assioma `stat`), espressa nella notazione del manuale di Lua, dove il metasimbolo `::=` tiene luogo di $\rightarrow$:

```
stat      ::=  varlist '=' explist

varlist   ::=  var  ',' varlist | var

explist   ::=  exp1 ',' explist | exp1

exp1      ::=  exp1 or  exp2   | exp2

exp2      ::=  exp2 and exp3   | exp3

exp3      ::=  exp3 '<'  exp4 | exp3 '==' exp4 | exp4

exp4      ::=  exp5 '..' exp4 | exp5      \{ right associative op. \}

exp5      ::=  exp5 '+'  exp6 | exp5 '-' exp6 | exp6

exp6      ::=  exp6 '*'  exp7 | exp6 '/' exp7 | exp7

exp7      ::=  not exp7 | '#' exp7 | '-' exp7 | exp8

exp8      ::=  exp9 '^'  exp8 | exp9      \{ right associative op. \}

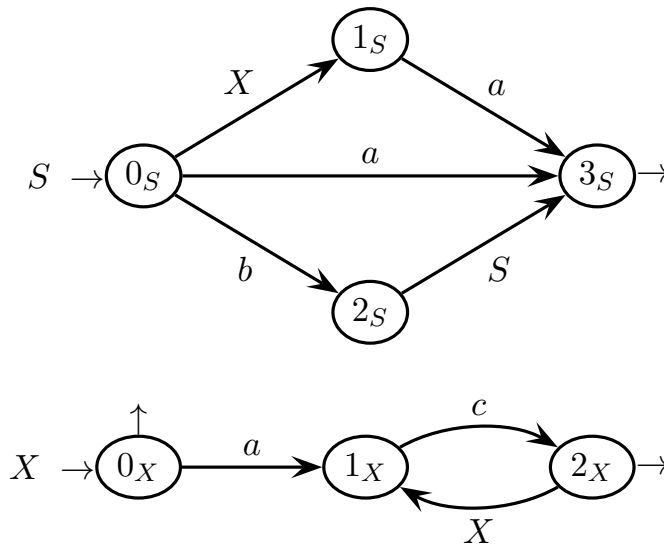
exp9      ::=  nil | false | true | Number | String | '(' exp1 ')'
```

Qui i nonterminali significativi della grammatica sono: `stat` (assioma), `varlist`, `explist` ed `exp[1-9]`; i nonterminali `var`, `Number` e `String` andrebbero espansi ulteriormente, ma non è richiesto; i simboli rimanenti `or`, `and`, `not`, `nil`, `false`, `true` e tutti quelli tra apici, sono terminali. Naturalmente si potrebbe esprimere la grammatica nella notazione usuale.

Si noti come i due operatori associativi a destra, ossia concatenamento `'..'` ed esponenziazione `'^'`, siano generati da regole ricorsive a destra, mentre tutti gli altri hanno regole ricorsive a sinistra poiché sono associativi a sinistra. Per il resto la grammatica ha una struttura piuttosto standard e riprende la nota grammatica non ambigua delle espressioni aritmetiche, ampliandola per contemplare tutti gli operatori previsti da Lua e ordinando le regole ricorsive secondo le precedenze tra operatori.

3 Analisi sintattica e parsificatori 20%

1. È data la grammatica estesa seguente, rappresentata come rete di macchine ricorsive (assioma S):



Si risponda alle domande seguenti:

- (a) Si tracci il grafo pilota della rete e si dica se essa è di tipo *ELR*, elencando gli eventuali conflitti e precisandone il tipo.
- (b) Si tracci il grafo di controllo *PCFG* della rete con gli insiemi di prospezione (negli stati finali) e guida (sugli archi di chiamata e sulle frecce di uscita), e si dica se esso è deterministico, elencando gli eventuali punti di nondeterminismo. Inoltre si dica se il grafo pilota soddisfa la condizione Single Transition Property (*STP*), motivando la risposta.
- (c) (facoltativa) La a da 1_S a 3_S diventa c e 0_X non è più finale; il resto è invariato. La rete è *ELR*? La rete è *ELL*? Si motivino le risposte sinteticamente.

PER QUANTO SEGUE VEDI ANCHE AVVISO SUL FRONTESPIZIO DEL TESTO

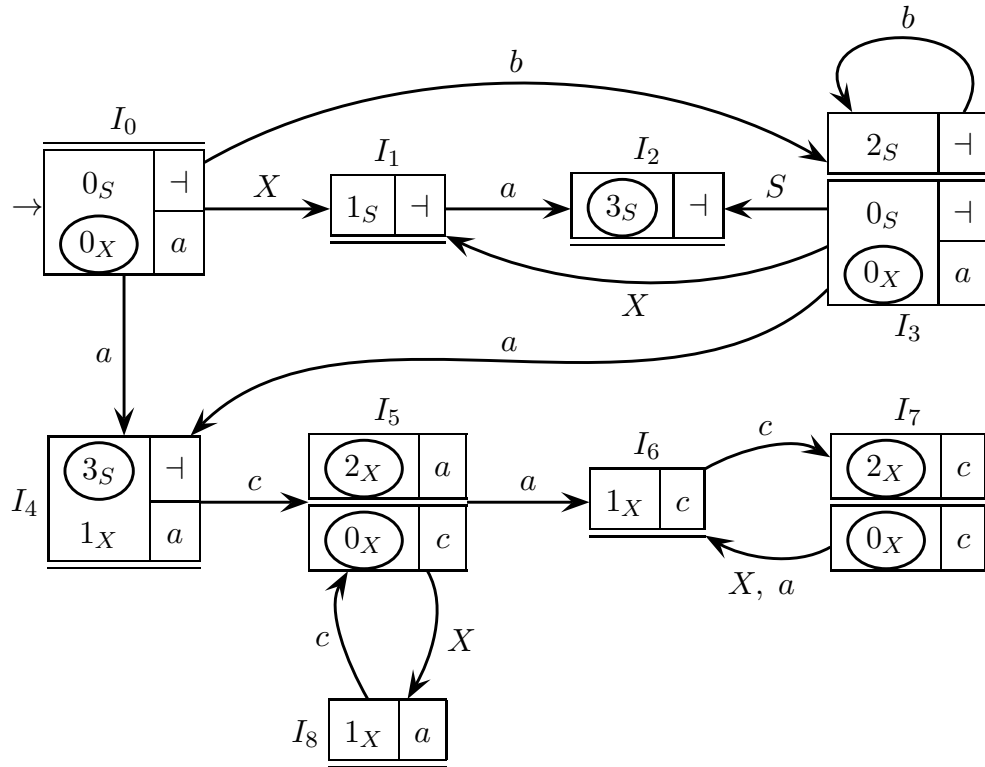
Chi preferisse rispondere ad (a-b) con la metodologia classica di analisi *LR* e *LL*, usi la grammatica non estesa seguente (assioma S), equivalente alla rete di macchine:

$$\left\{ \begin{array}{l} S \rightarrow X a \mid b S \mid a \\ X \rightarrow a c Y \mid \varepsilon \\ Y \rightarrow X c Y \mid \varepsilon \end{array} \right.$$

Per rispondere a (c) secondo la metodologia classica LR e LL , si riportino sulla grammatica non estesa le modifiche indicate.

Soluzione

(a) Ecco il grafo pilota *ELR* della rete, con distanza di look-ahead $k = 1$:

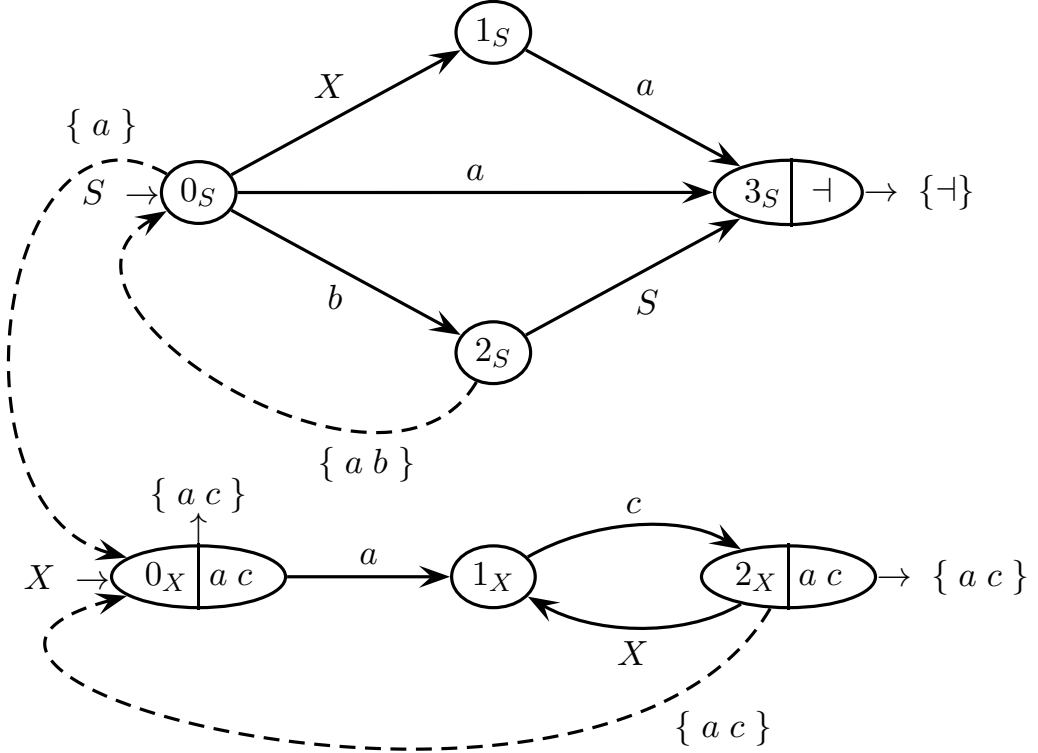


La rete non è di tipo *ELR*. Eccone i conflitti con descrizione:

- in I_0 : shift-reduce tra lo shift di a verso I_4 e la candidata $\langle \langle 0_X \rangle a \rangle$
- in I_3 : idem
- in I_5 : shift-reduce tra lo shift di a verso I_6 e la candidata $\langle \langle 2_X \rangle a \rangle$
- in I_7 : reduce-reduce tra le due candidate $\langle \langle 2_X \rangle c \rangle$ e $\langle \langle 0_X \rangle c \rangle$

La rete non ha conflitti di convergenza. Del resto la grammatica è ambigua: per esempio, la stringa a è riconosciuta in due modi.

(b) Ecco il grafo di controllo *PCFG*, con gli insiemi di prospezione e guida:



Si tenga presente che S non è nullabile e che X è nullabile. Il *PCFG* non è deterministico. Ecco i punti di nondeterminismo, con descrizione:

- in 0_S : tra shift $0_S \xrightarrow{a} 3_S$ e call $0_S \xrightarrow{b} 2_S$
- in 0_X : tra shift $0_X \xrightarrow{a} 1_X$ e return $0_X \xrightarrow{X} 0_S$
- in 2_X : tra return $2_X \xrightarrow{X} 1_X$ e call $2_X \xrightarrow{c} 0_X$

Comunque si ricordi che la grammatica è ambigua, dunque non è analizzabile deterministicamente.

La grammatica non ha la proprietà *STP* poiché il grafo pilota ha due m-transizioni multiple: $I_0 \xrightarrow{a} I_4$ e $I_3 \xrightarrow{a} I_4$; entrambe infatti comprendono due transizioni tra stati della rete, ossia $0_S, 0_X \xrightarrow{a} 3_S, 1_X$.

- (c) La rete diventa *ELR*: tutti i look-ahead a diventano c ; I_5 si identifica con I_7 ; I_6 con I_8 ; e di conseguenza tutti i conflitti elencati al punto (a) scompaiono. Si noti infatti che la stringa a non è più ambigua.

La rete NON diventa *ELL*: le m-transizioni multiple di cui al punto (b), restano.

Nota: le soluzioni secondo la teoria classica *LR* e *LL*, sono lasciate al lettore.

4 Traduzione e analisi semantica 20%

1. La grammatica G seguente (assioma S):

$$G: S \rightarrow (S) S \mid \varepsilon$$

definisce il linguaggio di Dyck con parentesi tonde ‘(’ e ‘)’. Si vuole uno schema di traduzione τ dove le parentesi tonde a livello di annidamento n ($n \geq 1$) siano traslitterate in parentesi graffe ‘{’ e ‘}’ se vale la relazione $n \bmod 3 = 1$, in parentesi quadre se $n \bmod 3 = 2$, mentre se $n \bmod 3 = 0$ le parentesi tonde rimangono invariate.

Nota: le parentesi tonde più esterne della stringa sorgente sono considerate a livello di annidamento 1 (si veda l’esempio).

Per esempio la stringa sorgente x :

$$x = (()) ((((()))))$$

viene tradotta nella stringa destinazione $\tau(x)$:

$$\tau(x) = \{ [] \} \{ [] [(\{ \})] \}$$

Si risponda alle domande seguenti:

- (a) Si modifichi la grammatica G data, ottenendo la grammatica sorgente G_s di uno schema di traduzione adatto a definire la traduzione τ descritta sopra.
- (b) Si completi lo schema di traduzione τ descritto sopra, aggiungendo la parte destinazione G_d alla grammatica sorgente G_s ottenuta prima.
- (c) (facoltativa) Si disegni l’albero sintattico della traduzione τ per la stringa sorgente x di esempio mostrata prima:

$$x = (()) ((((()))))$$

Soluzione

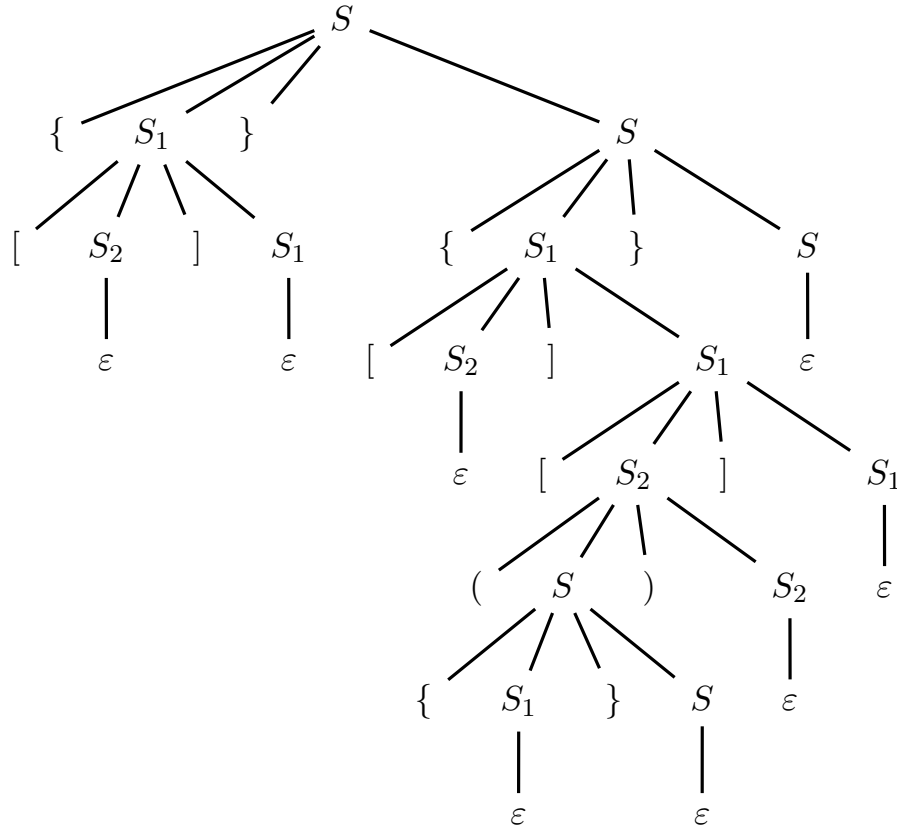
- (a) Per distinguere le parentesi ai diversi livelli di annidamento occorre introdurre due nuovi nonterminali, così da avere un nonterminale per ogni tipo di parentesi. Dunque la grammatica sorgente G_s di τ è la seguente (assioma S):

$$G_s \begin{cases} S \rightarrow (S_1) S \mid \varepsilon \\ S_1 \rightarrow (S_2) S_1 \mid \varepsilon \\ S_2 \rightarrow (S) S_2 \mid \varepsilon \end{cases}$$

- (b) La parte destinazione G_d di τ è la seguente (assioma S):

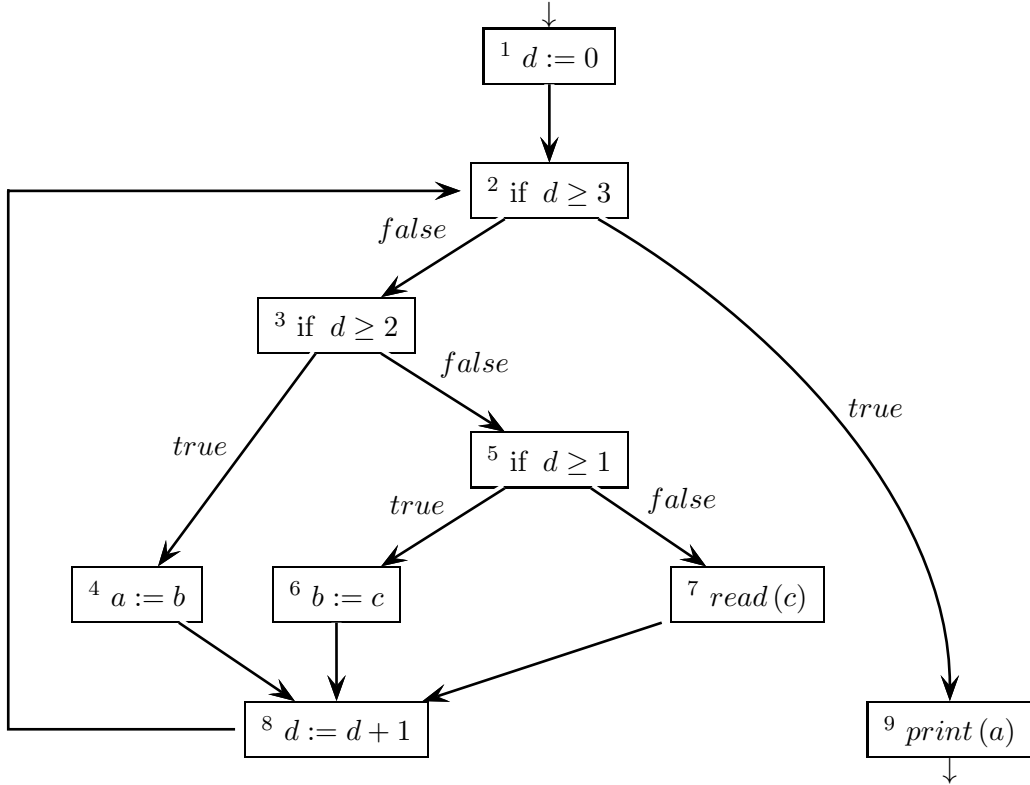
$$G_d \begin{cases} S \rightarrow \{ S_1 \} S \mid \varepsilon \\ S_1 \rightarrow [S_2] S_1 \mid \varepsilon \\ S_2 \rightarrow (S) S_2 \mid \varepsilon \end{cases}$$

- (c) Ecco l'albero sintattico destinazione della stringa x :



Naturalmente l'albero sorgente ha la stessa struttura, dove però le foglie terminali (non nulle) sono tutte parentesi tonde (aperte o chiuse).

2. Nel programma seguente, rappresentato dal suo Control-Flow Graph, si vogliono fare certe analisi statiche, indicate di seguito.



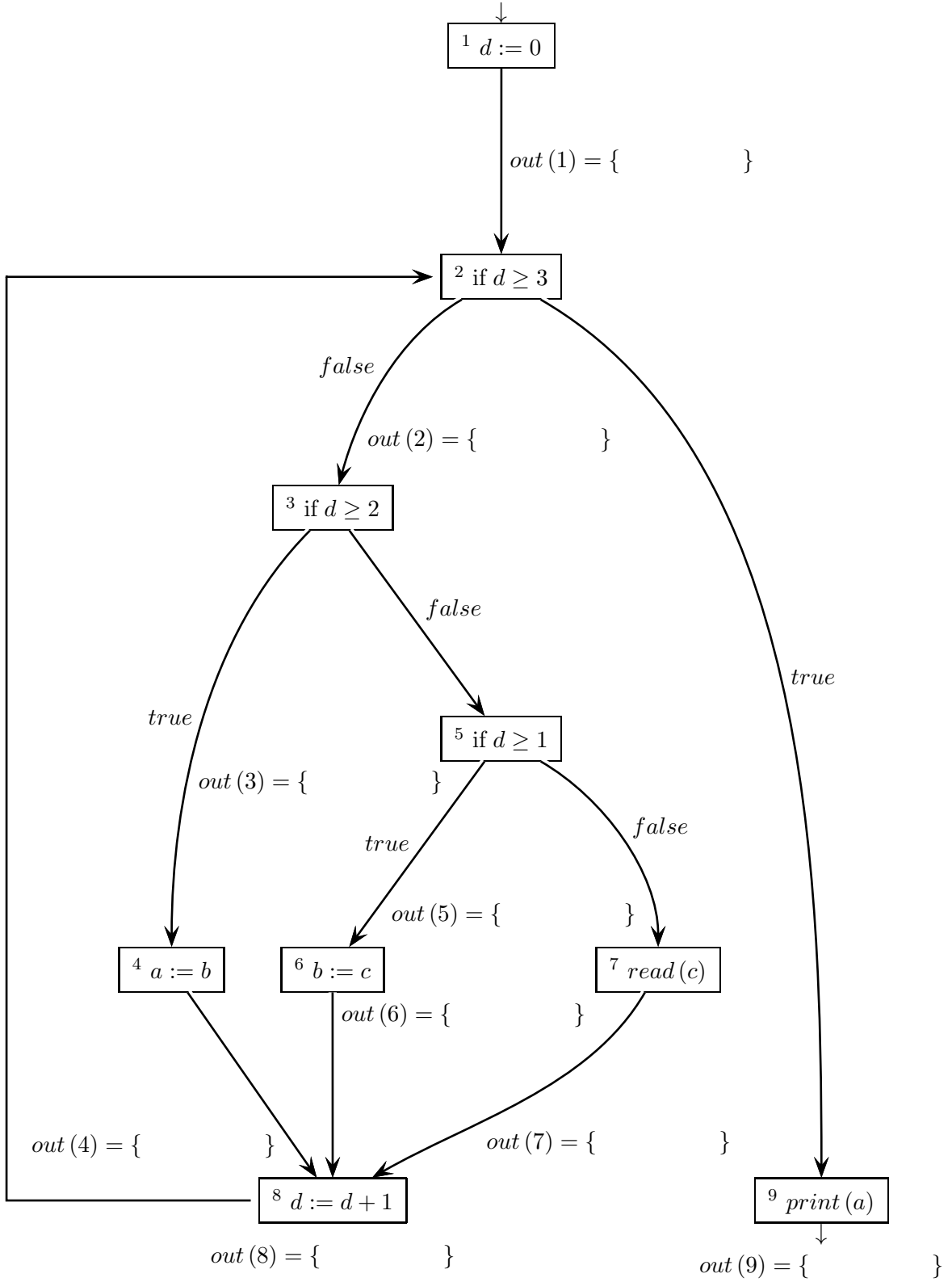
Si risponda alle domande seguenti:

- Si scrivano, nello spazio predisposto di seguito, le equazioni di flusso per calcolare le definizioni raggiungenti (reaching definitions) l'uscita del nodo 8 del grafo. Inoltre si calcolino tutti gli insiemi delle definizioni raggiungenti le uscite di tutti i nodi del grafo. Si scrivano tali insiemi nella figura predisposta di seguito.
- (facoltativa) Si vuole controllare se ci sono variabili *potenzialmente non inizializzate* (*Possibly Uninitialized* o *PU*). Una variabile x è *PU* in un punto u del grafo, se esiste un cammino dal nodo iniziale al nodo u lungo il quale:
 - non si trova una definizione della variabile x
 - oppure la definizione della variabile x è del tipo $x \leftarrow e$, dove l'espressione e usa una variabile *PU*

Per esempio la variabile b è *PU* nel punto *out*(4) poiché lungo il cammino 1, 2, 3, 4 non esiste una def. di b . Invece la variabile d non è *PU* in *out*(4) poiché ogni cammino da 1 a 4 contiene una def. di d . E la variabile b è *PU* in *out*(6) per via del cammino 1, 2, 3, 5, 6 contenente la def. $b \leftarrow c$ dove c risulta *PU*.

Nella tabella predisposta di seguito, si scrivano gli insiemi delle variabili *PU* all'ingresso e all'uscita di ciascun nodo.

scrivere nella figura gli insiemi delle definizioni raggiungenti



scrivere qui le equazioni d flusso richieste (per il nodo 8)

scrivere nella tabella gli insiemi delle variabili Potentially Undefined (PU)

<i>nodo</i>	<i>in</i>	<i>out</i>
1		
2		
3		
4		
5		
6		
7		
8		
9		

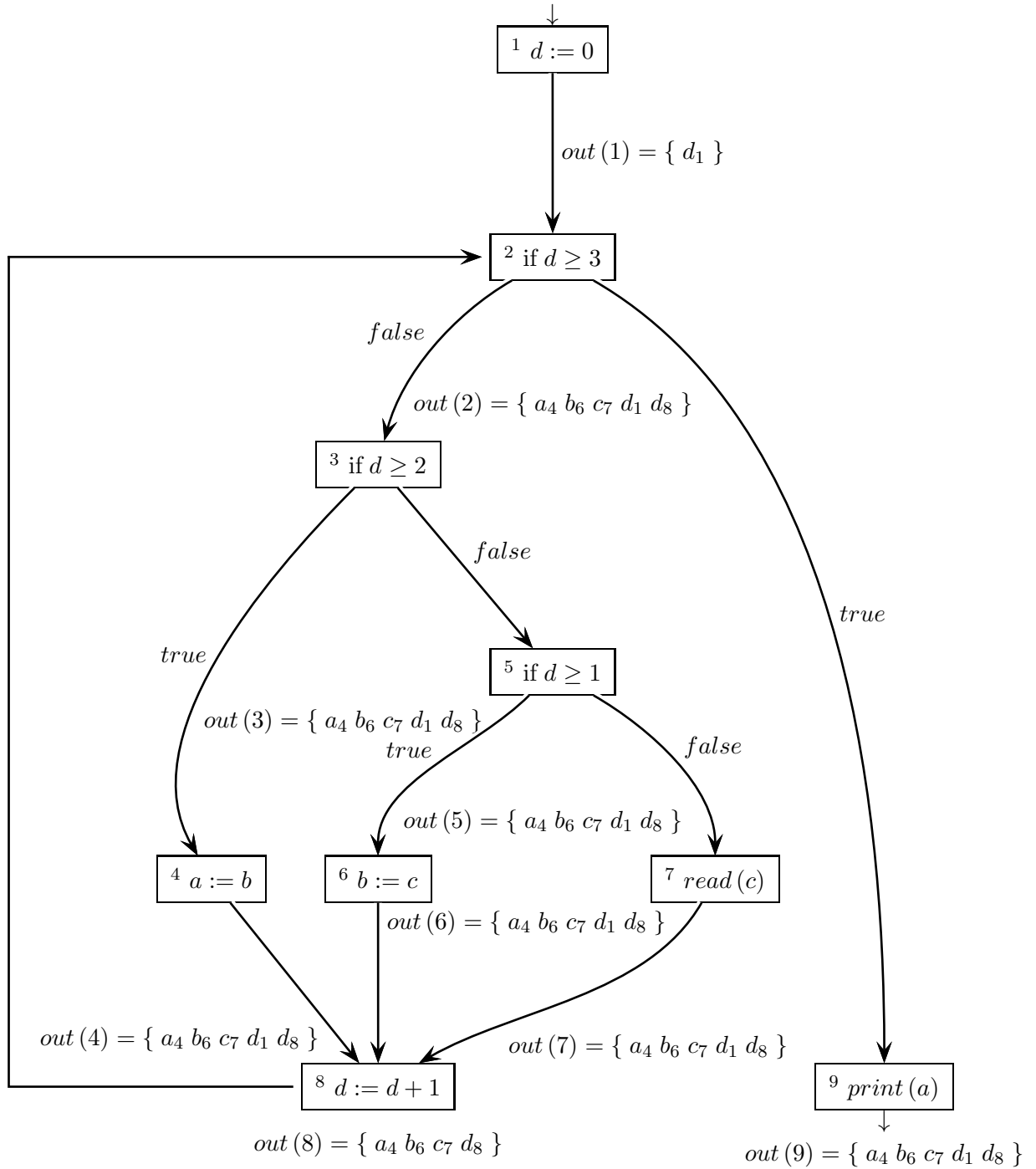
Soluzione

(a) Le equazioni di flusso del nodo 8 sono le seguenti:

$$in(8) = out(4) \cup out(6) \cup out(7)$$

$$out(8) = \{ d_8 \} \cup (in(8) \setminus \{ d_1 \})$$

Gli insiemi delle definizioni raggiungenti sono scritti nella figura:



- (b) Una variabile x è PU in un punto u del grafo, se esiste un cammino dal nodo iniziale al nodo u lungo il quale:
- i. non si trova una definizione della variabile x
 - ii. oppure la definizione di x è del tipo $x \leftarrow e$, dove l'espressione e usa una variabile PU

È possibile calcolare gli insiemi PU direttamente dalla definizione data, applicandola ai cammini del grafo. L'analisi delle variabili potenzialmente non inizializzate produce questi insiemi:

<i>nodo</i>	<i>in</i>	<i>out</i>
1	$a\ b\ c\ d$	$a\ b\ c$
2	$a\ b\ c$	$a\ b\ c$
3	$a\ b\ c$	$a\ b\ c$
4	$a\ b\ c$	$a\ b\ c$
5	$a\ b\ c$	$a\ b\ c$
6	$a\ b\ c$	$a\ b\ c$
7	$a\ b\ c$	$a\ b$
8	$a\ b\ c$	$a\ b\ c$
9	$a\ b\ c$	$a\ b\ c$

Si possono qui facilmente ritrovare gli esempi dati nel testo.